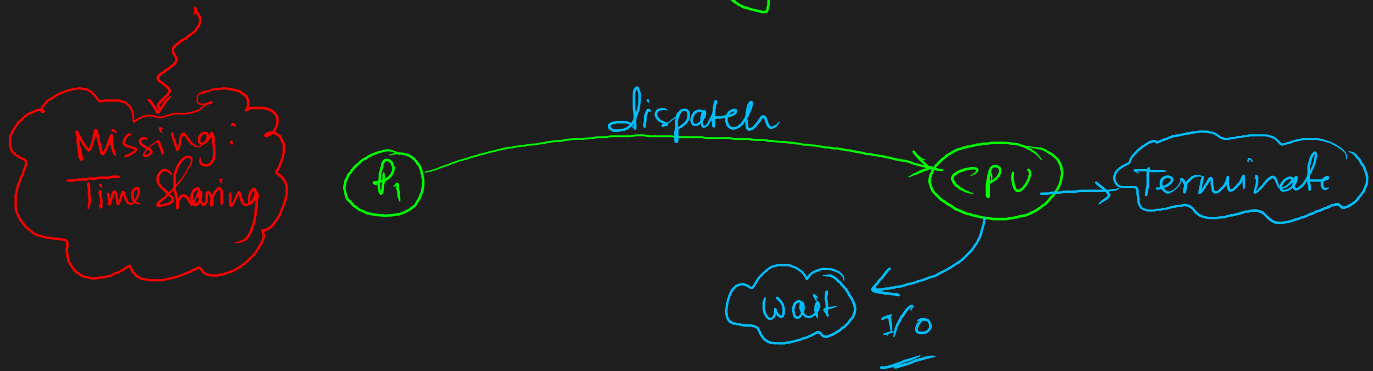


29/06/26

CPU Process Scheduling

Lee-13

① Non-preemptive Scheduling



② Preemptive Scheduling

→ Extra: Time Quantum Expire
↓
CPU Free

* Comparison

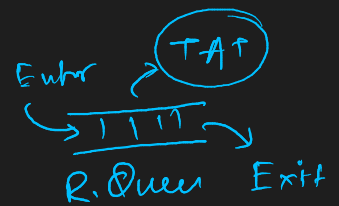
- ① Starvation: Non-preemptive ↑↑ preemptive ↓
- ② CPU utilization: Preemptive ↑
- ③ Overhead: preemptive ↑

* Goals of CPU scheduling Algorithms.

- ① Maximum CPU utilization
- ② Minimum TAT (Turn around Time)
- ③ Min. wait time (w.t) (CPU should not wait more)
- ④ Min Response time (RT) (Process comes to Ready Queue)

⑤ Max. throughput.

[#processes completed.
/ unit time.]



Response time.
↑
1st time got CPU

* AT (Arrival Time) → Time when process is arrived at Ready Queue.

* BT (Burst Time) → Time required to execute a process.

* CT (Completion Time) (if there is only 1 process)

↳ Time taken till process gets terminated

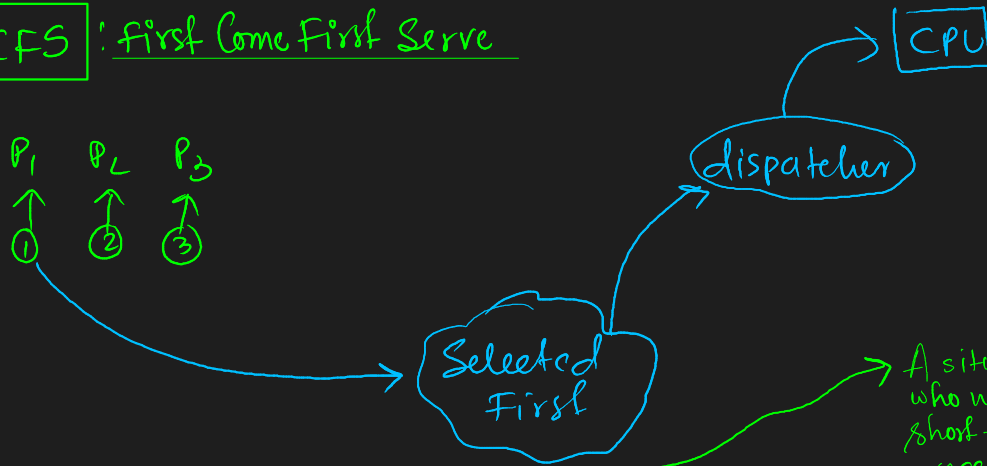
* $TAT = CT - AT$

* $WT = TAT - BT$

30/06/26

Scheduling Algorithms

① FCFS: First Come First Serve

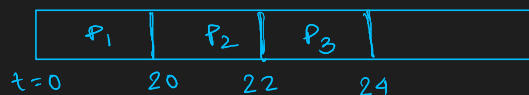


Convoy Effect

	Pno	AT	BT	CT	$CT - AT$ TAT	$TAT - BT$ WT
I	1	0	20	20	20	0
	2	1	2	22	21	19
	3	2	2	24	22	20

Cause poor Resource Management

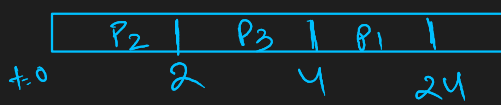
Ganttchart



Avg WT = 13 unit

	Pno	AT	BT	CT	TAT	WT
II	P2	0	2	2	2	0
	P3	1	2	4	3	1
	P1	2	20	24	22	2

GC



Avg = 1 ✓ unit

② SJF: Shortest Job First (Non-Preemptive)

- Process with least BT will get CPU.

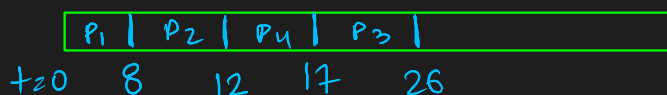
$P_1, P_2, P_3 \rightarrow$ BT??

P	AT	BT	CT	TAT	WT
P_1	0	8	8	8	0
P_2	1	4	12	11	7
P_3	2	9	26	24	15
P_4	3	5	17	14	9

Smaller

Avg WT = 7.75 unit.

NonPre



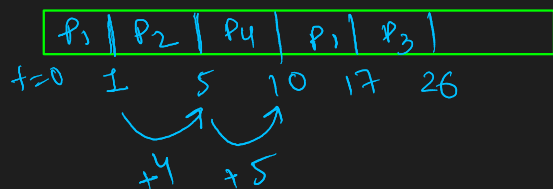
7/07/20

* Preemptive SJF

P	AT	BT	CT	TAT	WT
P_1	0	8 7	17	17	9
P_2	1	4	5	4	0
P_3	2	9	26	24	15
P_4	3	5	10	7	2

Avg WT = 6.5 unit.

GC



③ Priority Scheduling $\rightarrow$ Assign priority to each job.

* Non-preemptive

P	Priority	AT	BT	CT	TAT	WT
1	2	0	4	4	4	0
2	4	1	2	25	24	22
3	6	2	3	23	22	19
4	10	3	5	9	6	1
5	8	4	1	20	16	15
6	12	5	4	13	8	4
7	9	6	6	19	13	7



Avg WT = 9.71 unit

* Preemptive

Pno.	P*	AT	BT	CT	T-AT	WT
1	2	0	4/3	25	25	21
2	4	1	2/1	22	21	20
3	6	2	2/2	21	19	16
4	10	3	3/3	12	9	4
5	8	4	1	19	15	14
6	12	5	4	9	4	0
7	9	6	6	18	12	6

Arg. WT = 11.57

Unit

P ₁	P ₂	P ₃	P ₄	P ₆	P ₁	P ₇	P ₅	P ₃	P ₂	P ₁
t=0	1	2	3	5	9	12	18	19	21	22 25

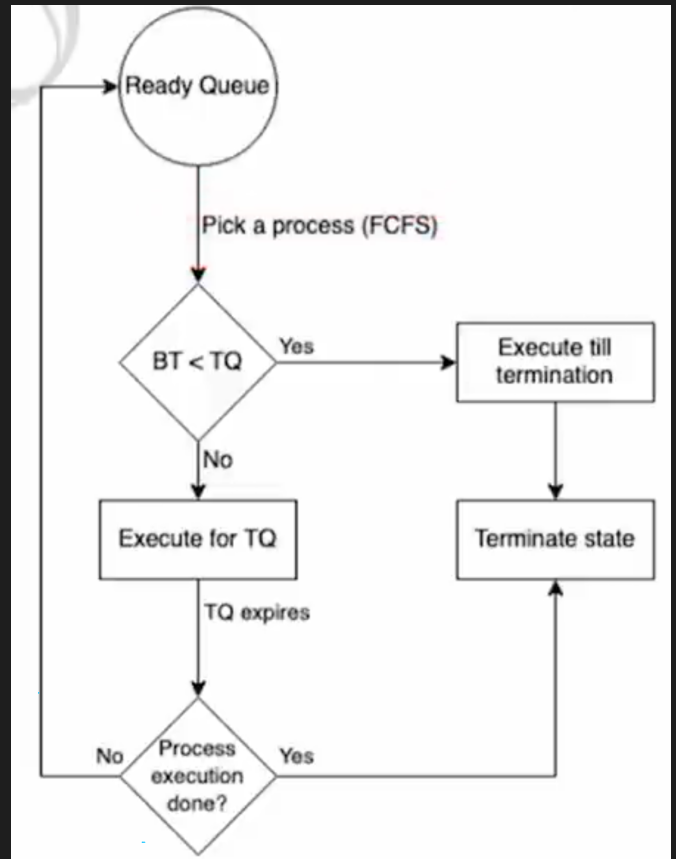
Biggest Drawback → Both P & Non-P

① Indefinite waiting or extreme starvation

↓ Solution → Ageing → gradually increase priority of lowest priority jobs.

④ Round Robin (RR)

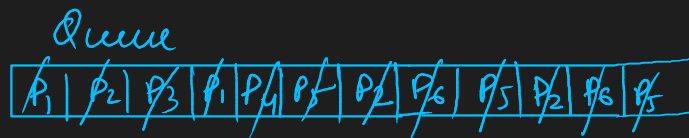
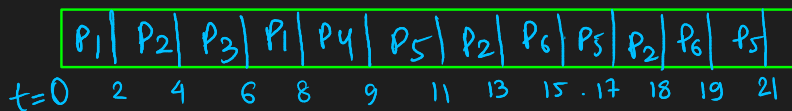
- Most popular
- Preemptive version of FCFS.
- Criteria: $AT + TQ$
 $\downarrow$ Time Quantum.
- design: Time sharing.
- Easy to implement.



P	AT	BT
1	0	4 2 0
2	1	5 3 10
3	2	2 0
4	3	1 0
5	4	6 4 7 0
6	6	2 1 0

$TQ \uparrow \Rightarrow \text{Overhead} \downarrow \Rightarrow \text{Context Switching} \downarrow$
 $TQ \downarrow \Rightarrow \text{Overhead} \uparrow \Rightarrow \text{CS} \uparrow$

$TQ = 25$



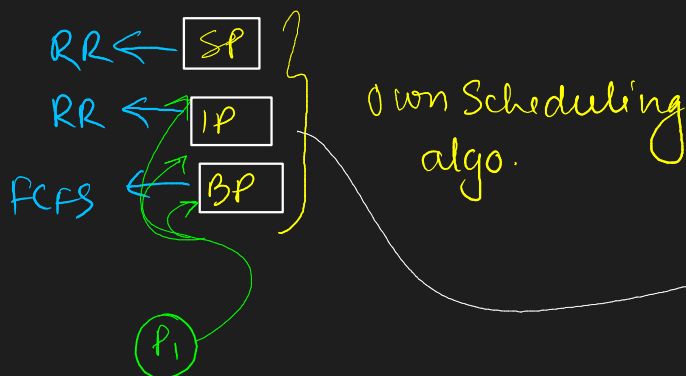
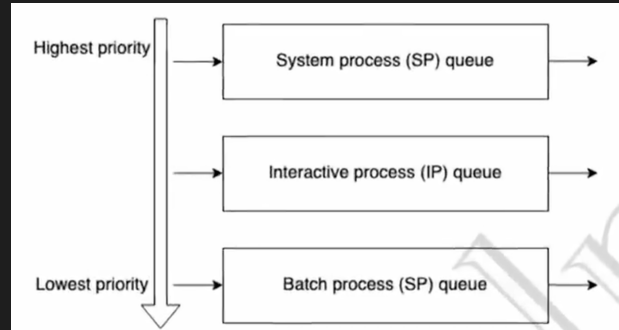
lec-14

(MLQ)

⑤ Multi Level Queue Scheduling :

- System Process : Created by OS.
- Interactive Process : user input. required.
(Foreground Process)
- Batch Process : No user input.
(Background)

Eg.



Convoy Effect $\rightarrow$ Present.

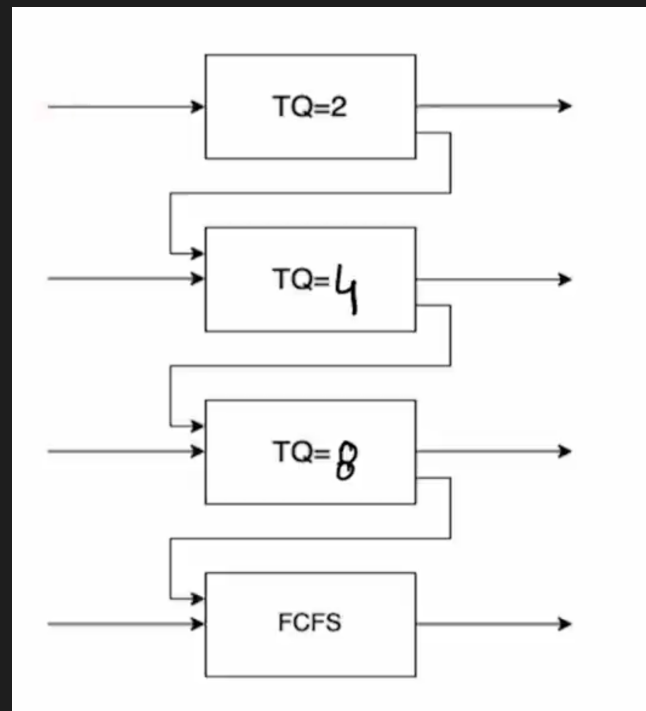
Schedules ??

$SP > IP > BP$

⑥ Multi Level Feedback Queue Scheduling (MLFQ)

- Multiple Subqueues.
- * Inter queue movement^{on} of the process is $\checkmark$.
- Separate processes based on BT.

- BT $\uparrow \Rightarrow$ Move to Lower Queue.
- I/O Bound & Interactive process higher priority.
- Ageing Method apply to lower priority processes.



Suppose

RR

RR

RR

FCFS.

Design of MLFQ.

- ① No. of Queue.
- ② Scheduling algo in each queue.
- ③ Method to upgrade a process to a higher queue.
- ④ Demote a process to lower process.
- ⑤ $P_i \rightarrow$ in which queue it will be pushed.

Comparisons of all algorithms. ✓

3. Comparison:

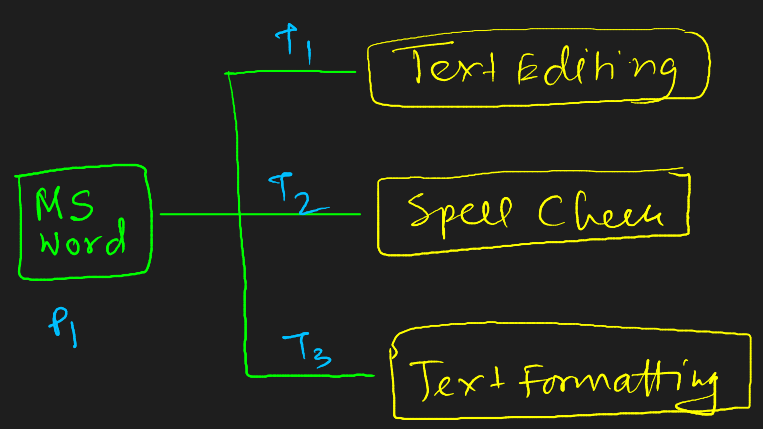
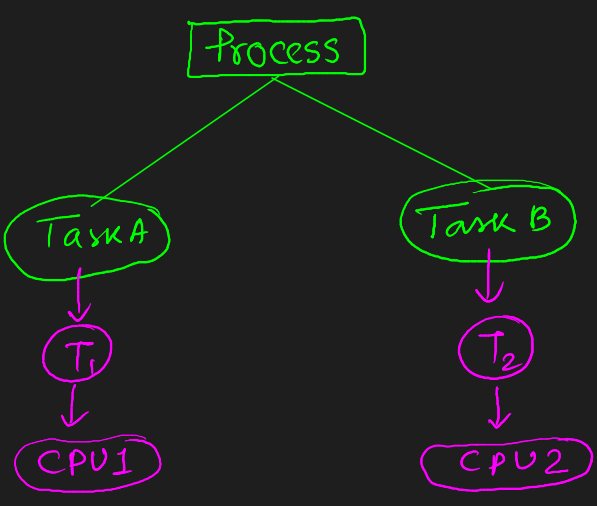
	FCFS	SJF	PSJF	Priority	P-Priority	RR	MLQ	MLFQ
Design	Simple	Complex	Complex	Complex	Complex	Simple	Complex	Complex
Preemption	No	No	Yes	No	Yes	Yes	Yes	Yes
Convoy effect	Yes	Yes	No	Yes	Yes	No	Yes	Yes
Overhead	No	No	Yes	No	Yes	Yes	Yes	Yes

8/07/26

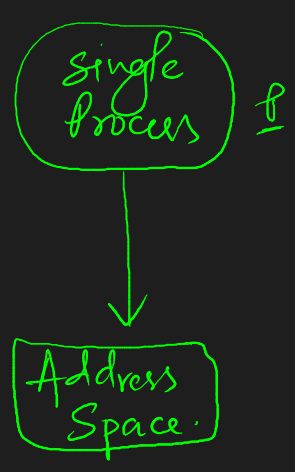
Lec-15

Concurrency

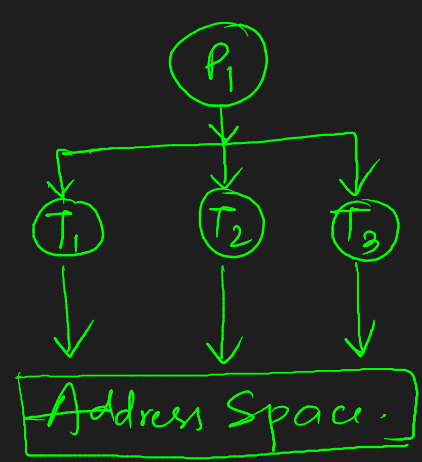
→ Multiple instructions at a single time.



Process

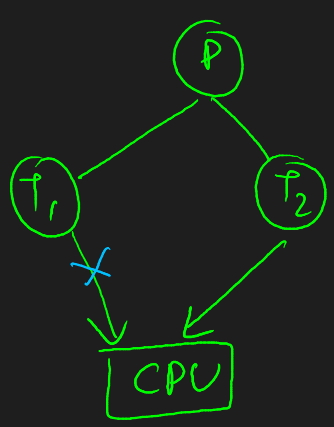


Thread



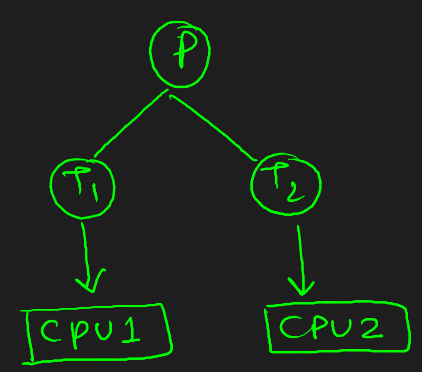
PCB
(Program Counter Block.)
— slow context switching.

TCB
(Thread Counter Block.)
— Very fast context switching.



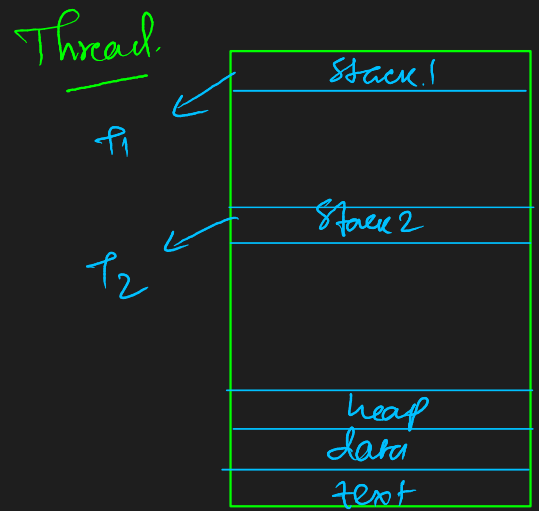
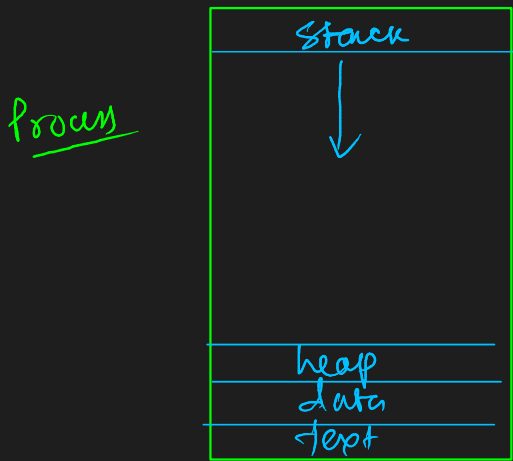
Sequential.

use > 1 CPU



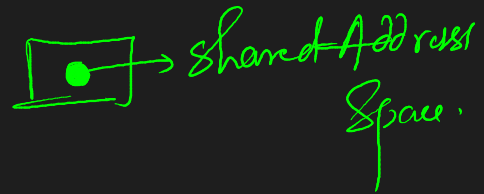
Parallel.

Mult Core CPU



Benefits of Multithreading

- ① Responsiveness → interactive.
- ② Resource Sharing.
- ③ Economy.
- ④ Threads better utilizes multicore cpu.



thread_python.py

```

1  import time
2  import threading
3
4  def taskA() → None:
5      for i in range(10):
6          time.sleep(1)
7          print(f"TaskA: {i}", flush=True)
8
9  def taskB() → None:
10     for i in range(10):
11         time.sleep(1)
12         print(f"TaskB: {i}", flush=True)
13
14  def main() → None:
15     t1: Thread = threading.Thread(target=taskA)
16     t2: Thread = threading.Thread(target=taskB)
17
18     t1.start()
19     t2.start()
20
21     t1.join()
22     t2.join()
23
24  if __name__ == "__main__":
25     main()
26

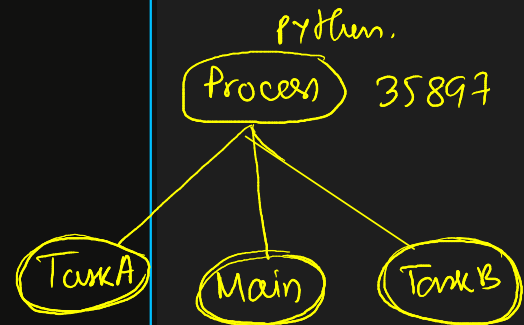
```

python3 thread_python.py

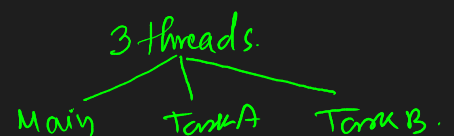
```

TaskB: 0
TaskA: 0
TaskA: 1
TaskB: 1
TaskB: 2
TaskA: 2
TaskB: 3
TaskA: 3
TaskB: 4
TaskA: 4
TaskA: 5
TaskB: 5
TaskA: 6
TaskB: 6
TaskA: 7
TaskB: 7
TaskA: 8
TaskB: 8
TaskA: 9
TaskB: 9

```



Pid:	Program:	Command:	Threads:	User:	MemB	Cpu%
35897	Python	/Library/Frameworks/Python.framework	3	anikchand	10M	0.0



lec-16

$count++ \Rightarrow count = count + 1$

$\Downarrow$
 $temp = count + 1$
 $count = temp$

Race Condition



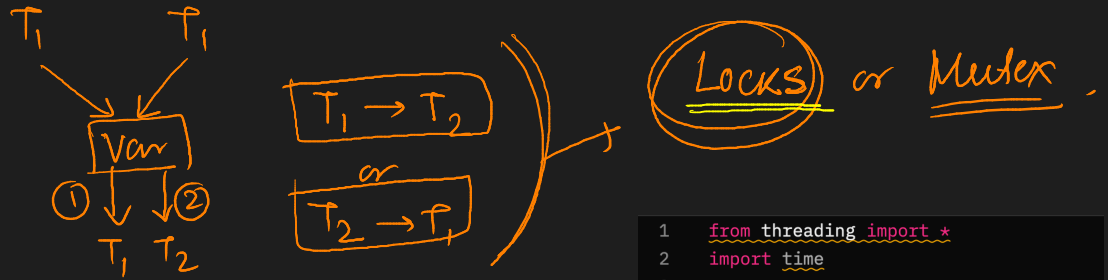
```
▲ /usr/bin/python3 criticalSectionProblem.py
1483519
▲ /usr/bin/python3 criticalSectionProblem.py
1550097
▲ /usr/bin/python3 criticalSectionProblem.py
1608715
▲ /usr/bin/python3 criticalSectionProblem.py
1417014
▲ /usr/bin/python3 criticalSectionProblem.py
1266120
▲ /usr/bin/python3 criticalSectionProblem.py
1446679
```

The result should be 20,00,000
but because of the Race condition
the results are coming < 20,00,000
& inconsistent.

```
1 from threading import *
2 import time
3
4
5 count = 0
6
7 def task() -> None:
8
9
10 global count
11
12 for i in range(1000000):
13     count += 1
14
15
16
17 if __name__ == "__main__":
18     t1: Thread = Thread(target=task)
19     t2: Thread = Thread(target=task)
20
21     t1.start()
22     t2.start()
23
24     t1.join()
25     t2.join()
26
27     print(count)
```

Solution of Race Condition

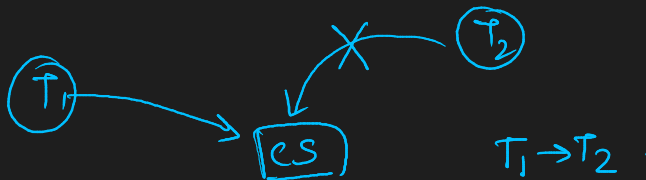
- ① Atomic operating variable $\rightarrow$ executes in a single CPU cycle.
(in C++ lang)
- ② Mutual Exclusion



```
▲ /usr/bin/python3 criticalSectionProblem.py
2000000
▲ /usr/bin/python3 criticalSectionProblem.py
2000000
▲ /usr/bin/python3 criticalSectionProblem.py
2000000
▲ /usr/bin/python3 criticalSectionProblem.py
2000000
▲ /usr/bin/python3 criticalSectionProblem.py
2000000
```

```
1 from threading import *
2 import time
3
4 lock: LockType = Lock()
5 count = 0
6
7 def task() -> None:
8     lock.acquire()
9
10    global count
11
12    for i in range(1000000):
13        count += 1
14
15    lock.release()
16
17 if __name__ == "__main__":
18     t1: Thread = Thread(target=task)
19     t2: Thread = Thread(target=task)
20
21     t1.start()
22     t2.start()
23
24     t1.join()
25     t2.join()
26
27     print(count)
```

③ Semaphores



Solⁿ of C.S should have 3 conditions $\rightarrow$

- ① Mutual Exclusion
- ② Progress



- ③ Bounded Waiting

$T_1 \rightarrow T_2$ X
 $T_2 \rightarrow T_1$ X
} No Fixed Order.

$\rightarrow$ indefinite waiting X
 $\rightarrow$ limited wait time $\checkmark$

Q. Can we use single flag?

T₁ thread

```
while True:
    while turn != 0:
        pass

    # C.S
    turn = 1

    # R.S
```

turn = flag 20/1

T₂ thread

```
while True:
    while turn != 1:
        pass

    # C.S
    turn = 0

    # R.S
```

turn = 0 ⇒ T₁ → T₂ → T₁ → T₂ → ...

turn = 1 ⇒ T₂ → T₁ → T₂ → T₁ → ...

↳ Fixed Order → Progress X

* Peterson Solution - flag[2] → [-, -]
- turn

T₁

```
while True:
    flag[0] = True
    turn = 1

    while turn == 1 and flag[1] == True:
        pass

    # C.S

    flag[0] = False
```

T₂

```
while True:
    flag[1] = True
    turn = 0

    while turn == 0 and flag[0] == True:
        pass

    # C.S

    flag[1] = False
```

flag:

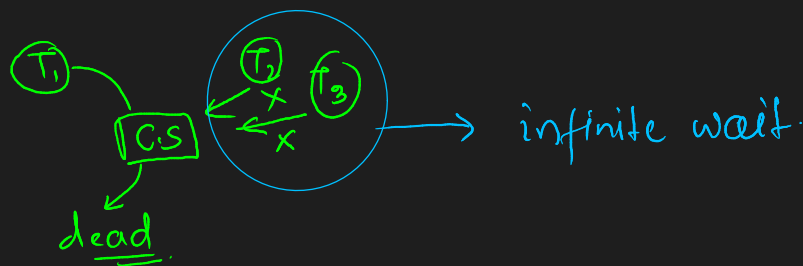
0	1

turn:

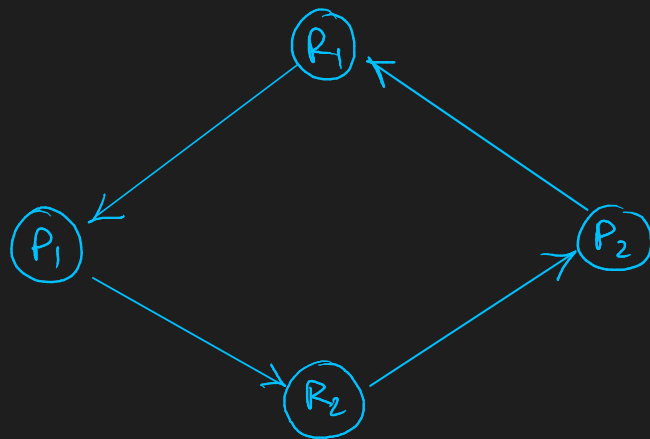
Peterson's soln - only thread safe in the case of 2 threads.

Locks disadvantages.

① Contention



② Dead Lock.



③ Debugging problem

~~~~~> complys because the operations are simultaneous.

④ Starvation →

